APRIL-MAY 2026

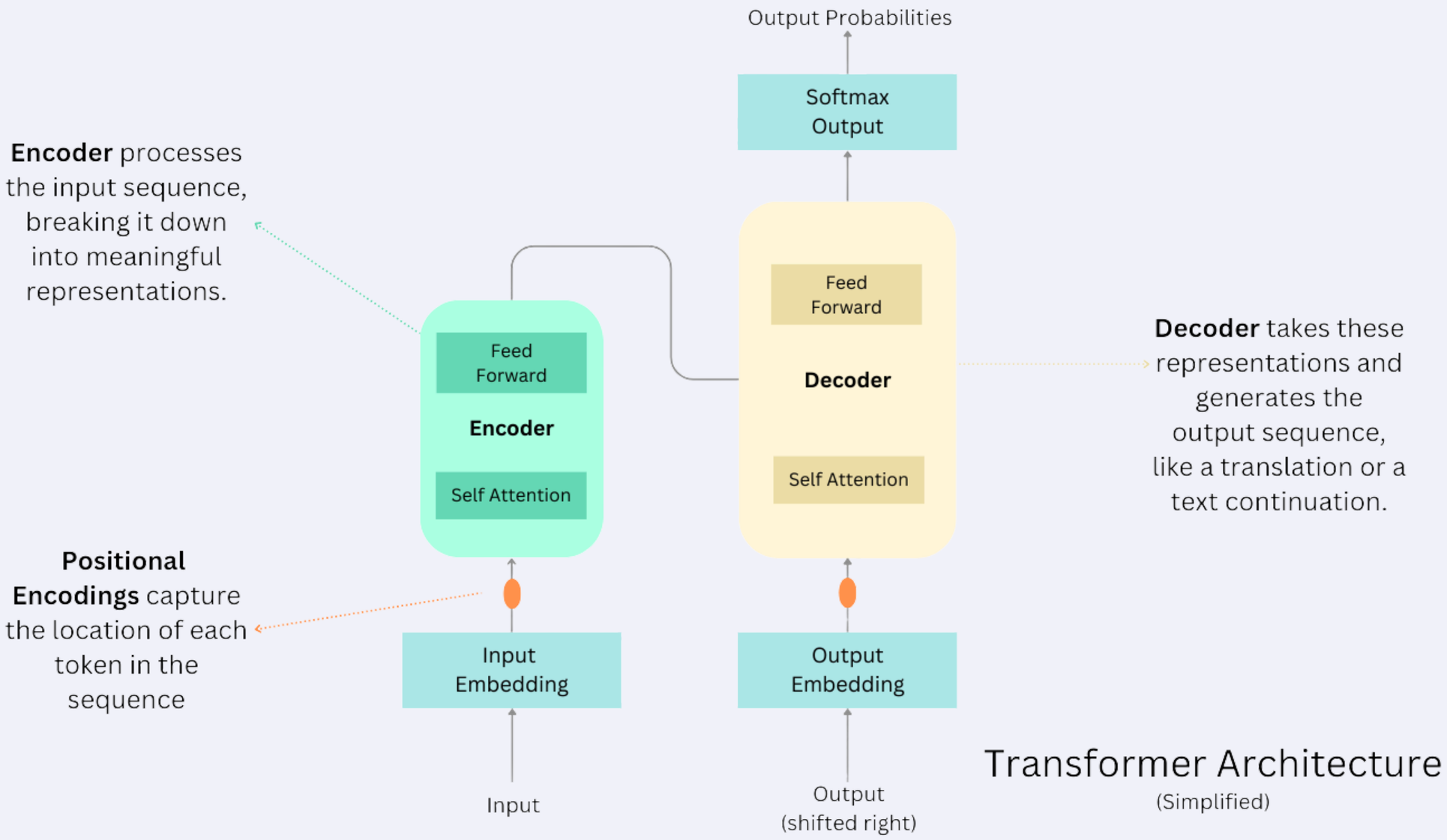
Large Language Models & Generative AI

From Pytorch to Frameworks

Giulia Coucourde

University of Turin · giulia.coucourde@unito.it

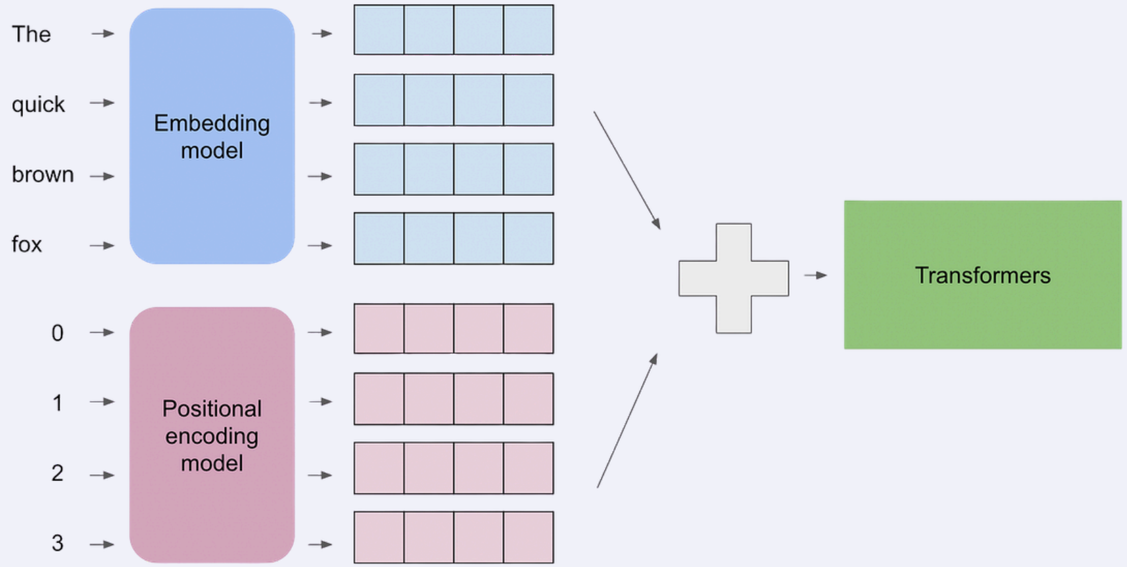
Transformer architecture



Transformer architecture

Embedding Layer

- Maps each input token to a dense vector representation.
- Learned during pre-training through gradient descent.
- Captures semantic and syntactic regularities of the language.



Positional Encoding

- Injects information about token order, which transformers cannot infer on their own.
- Allows the model to distinguish between sequences like “dog bites man” and “man bites dog.”
- Added to token embeddings before self-attention.

Input to Transformer

The resulting vectors, now enriched with positional information, are fed into the Transformer.

Attention Mechanism

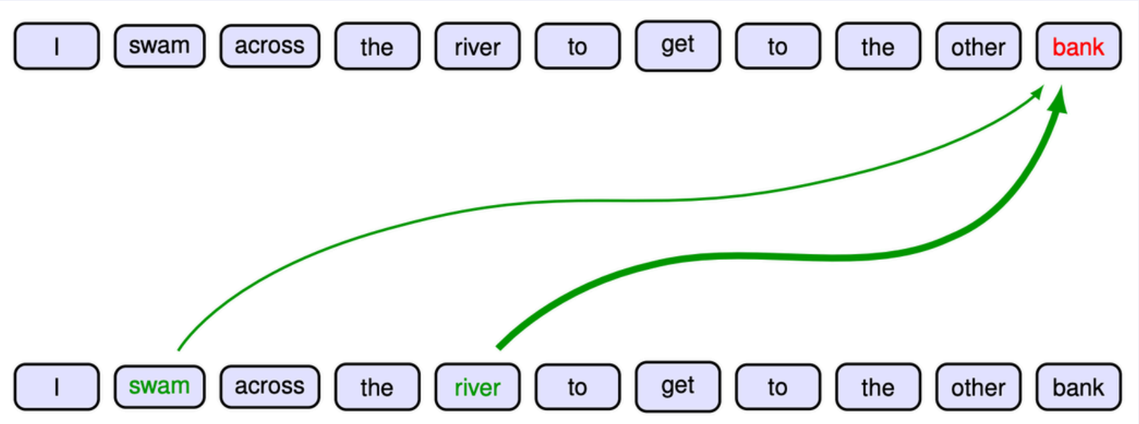
Originally developed as an enhancement to RNNs for machine translation: it allowed models ***to attend*** to specific words rather than compressing the whole sentence into one vector (RNN Encoder-Decoder).

*Attention is a processing mechanism that lets a network assign different importance weights to different inputs, where the weights themselves are computed **dynamically** from the input values.*

Weights in a standard neural network, once trained, are fixed!

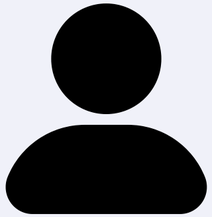
What to *attend*?

I swam across the river to get to the other bank.
I walked across the road to get cash from the bank.



Self-attention: intuitively

Consider the task of choosing a movie on a streaming platform: each movie is described by a set of attributes and linked to its content file, while you, as the user, have your own preferences over those attributes.



User preferences

[science-fiction drama, space exploration, hard-sci-fi elements]

Attributes

[science-fiction drama, space exploration epic, hard-sci-fi elements]

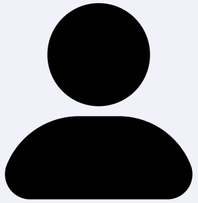
Attributes

[animated musical, family adventure, sisterhood narrative]

Attributes

[psychological horror, splatter, torture-based traps]

Self-attention: intuitively



User preferences

[science-fiction drama, space exploration, hard-sci-fi elements]

QUERY

VALUE

Attributes

[science-fiction drama, space exploration epic, hard-sci-fi elements...]

KEY

Attributes

[animated musical, family adventure, sisterhood narrative..]

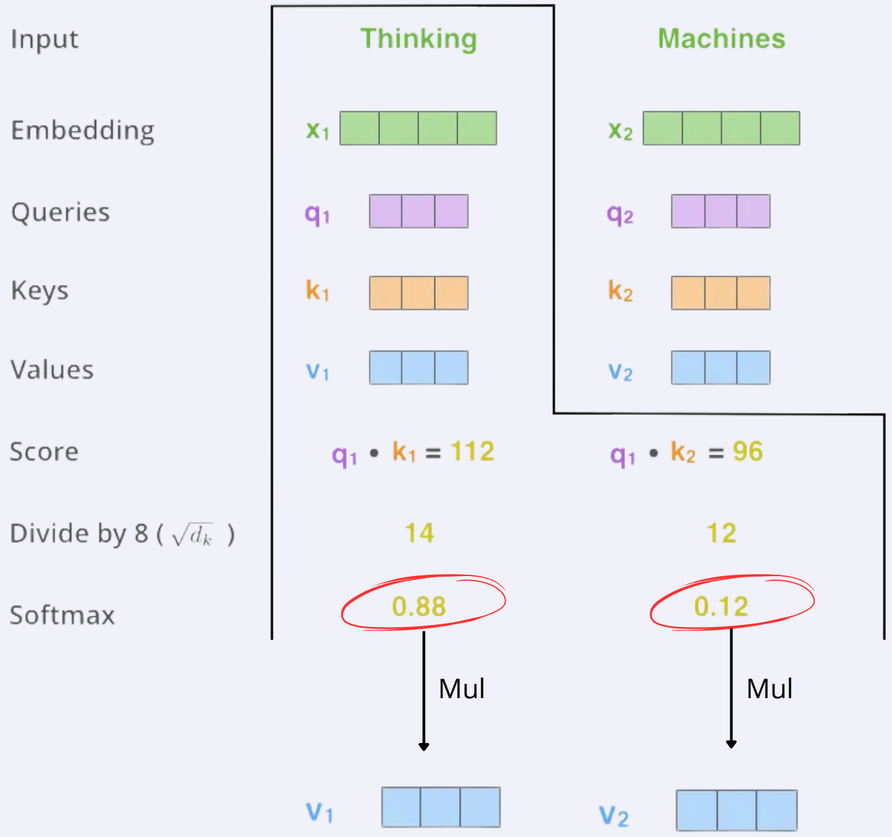
Attributes

[psychological horror, splatter, torture-based traps...]

The platform compares the user's query vector with all movie key vectors to find the closest match and returns the corresponding value file. In this analogy, the user *attends* to the movie whose key best aligns with the query. This is an example of **hard attention,** where a single value is selected.

Self-attention as soft attention

In transformers, the previous idea is extended to **soft attention**: instead of a single discrete match, we compute ***continuous measures*** of compatibility between queries and keys, and use these scores to weight the contribution of the value vectors to the output.



1. For an input sequence, **each token** produce

↓

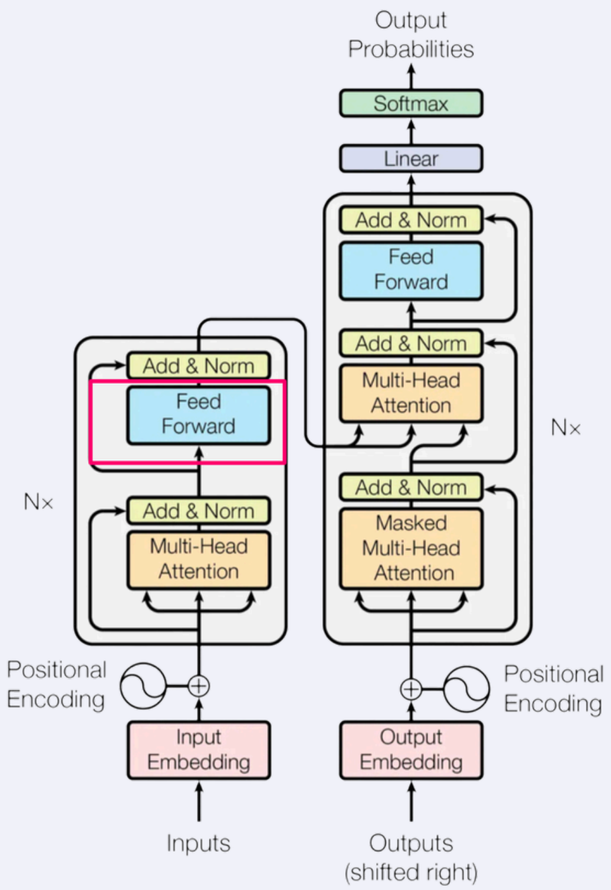
QUERY: what it is looking for
KEY: what it represents
VALUE: the information it carries

2. **Attention weight matrix.** For each token we calculate Q×KT: each token's query is compared to the keys of all tokens in the sequence.

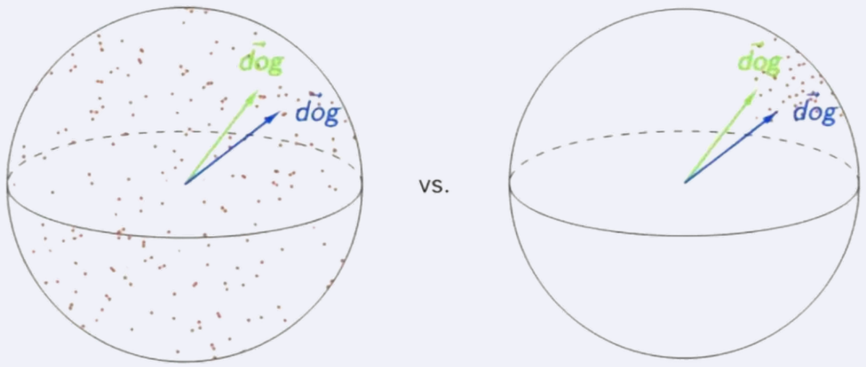
3. Once the attention weights are computed, we use them to combine the value vectors and to obtain the final output for the token.

Feed Forward Network

If attention decides *who to attend to*, the feed-forward decides *what to do* with that information.
FFN is inside every transformer layer and without it transformer learn nothing. But why?



FFN enhance the expressive power of transformer by introducing **non-linear transformations** at each token position trough **activation functions**.

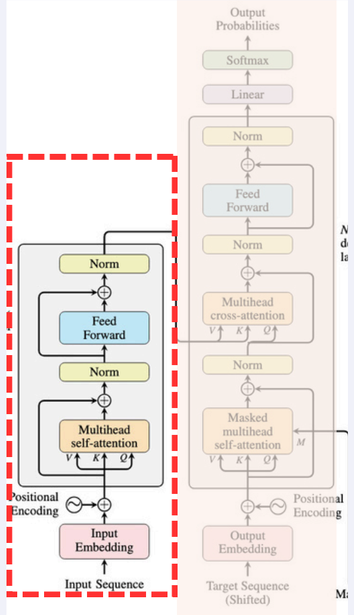


Right: token vectors are clustered in a narrow region (uniformity). This distribution belongs to the self-attention mechanism which is linear.
Goal: enhance representation diversity in high dimensional space.

Language Models

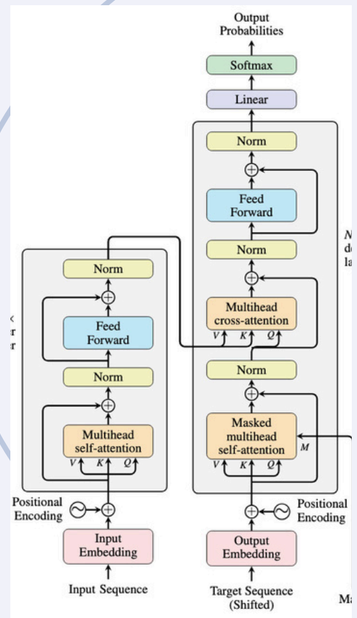
LLM

Encoder-only



Processes the entire input simultaneously using bidirectional self-attention, building a contextual representation of each token.
Ideal for **classification, named entity recognition, and semantic search**.

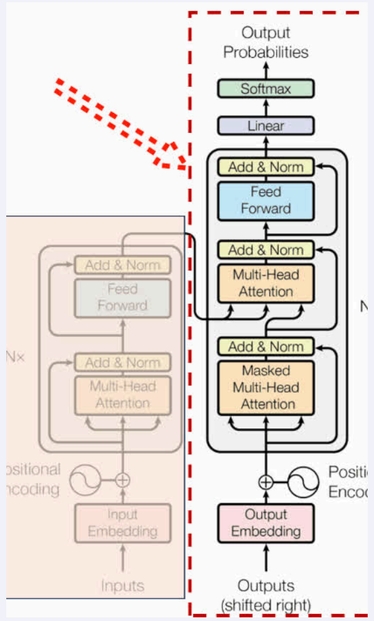
Encoder-decoder



Processes the entire input using the encoder, then generates an output with the decoder based on that representation.

Ideal for **translation or summarization.**

Decoder-only



Uses a single autoregressive stack that generates each token based on all previous tokens, treating both the input and the output as one continuous sequence.

Ideal for **open-ended text generation**

ENCODER-ONLY

Main architectures: BERT, ROBERTA, ALBERT, DistilBERT...

All share the same encoder-only structure, but they differ in training strategy, size, and efficiency.

Input tokens
↓
Embedding + Positional Encoding
↓
Transformer Encoder × N
(bidirectional self-attention + feedforward)
↓
[CLS] token
↓
Classifier
↓
Output label

WHEN TO USE:

- **Text classification:** sentiment, topic, spam detection
- **Named Entity Recognition:** people, places, dates
- **Semantic search:** find similar documents
- **Extractive QA:** find the answer span in a text

- ✗ **Text generation:** use decoder-only
- ✗ **Translation / summarization:** use encoder-decoder

ENCODER-ONLY: example

Each Transformer layer contains two sub-layers, repeated N times:

Layer 1:
self-attention
feedforward
↓
Layer 2:
self-attention
feedforward
↓
...
Layer N:
self-attention
feedforward

Why repeat the same block?

Given the following sentence: *"I really did not enjoy the film"*

Layer 1 → local syntactic relations
"not" is close to "enjoy"

Layer 2 → semantic relations
"not enjoy" carries negative meaning

Layer 3 → global context
the entire sentence expresses a negative opinion about the film

The number of layers is an **empirical choice:** more layers capture more complex patterns, but require more data, memory, and training time.

ENCODER-DECODER

Main architectures: T5, BART, mBART...

All share the same encoder-decoder structure, but they differ in training strategy, size, and efficiency.

Input sequence
↓
Embedding + Positional Encoding
↓
Transformer Encoder × N
(bidirectional self-attention + feedforward)
↓
Encoded representation
↓
Transformer Decoder × N
(masked self-attention + cross-attention + feedforward)
↓
Output token (one at a time)

WHEN TO USE:

- **Summarization:** condense long documents
- **Translation:** English → Italian
- **Question answering:** generative (not extractive)

- ❌ **Text generation:** use decoder-only
- ❌ **Text classification:** use encoder-only

ENCODER-DECODER: example

What makes encoder-decoder different from encoder-only? The decoder has THREE sub-layers, repeated N times:

Layer 1:
Masked self-attention → looks only at previous output tokens
Cross-attention → looks at the encoder output
Feedforward network
↓
Layer 2:
Masked self-attention
Cross-attention
Feedforward network
↓
...

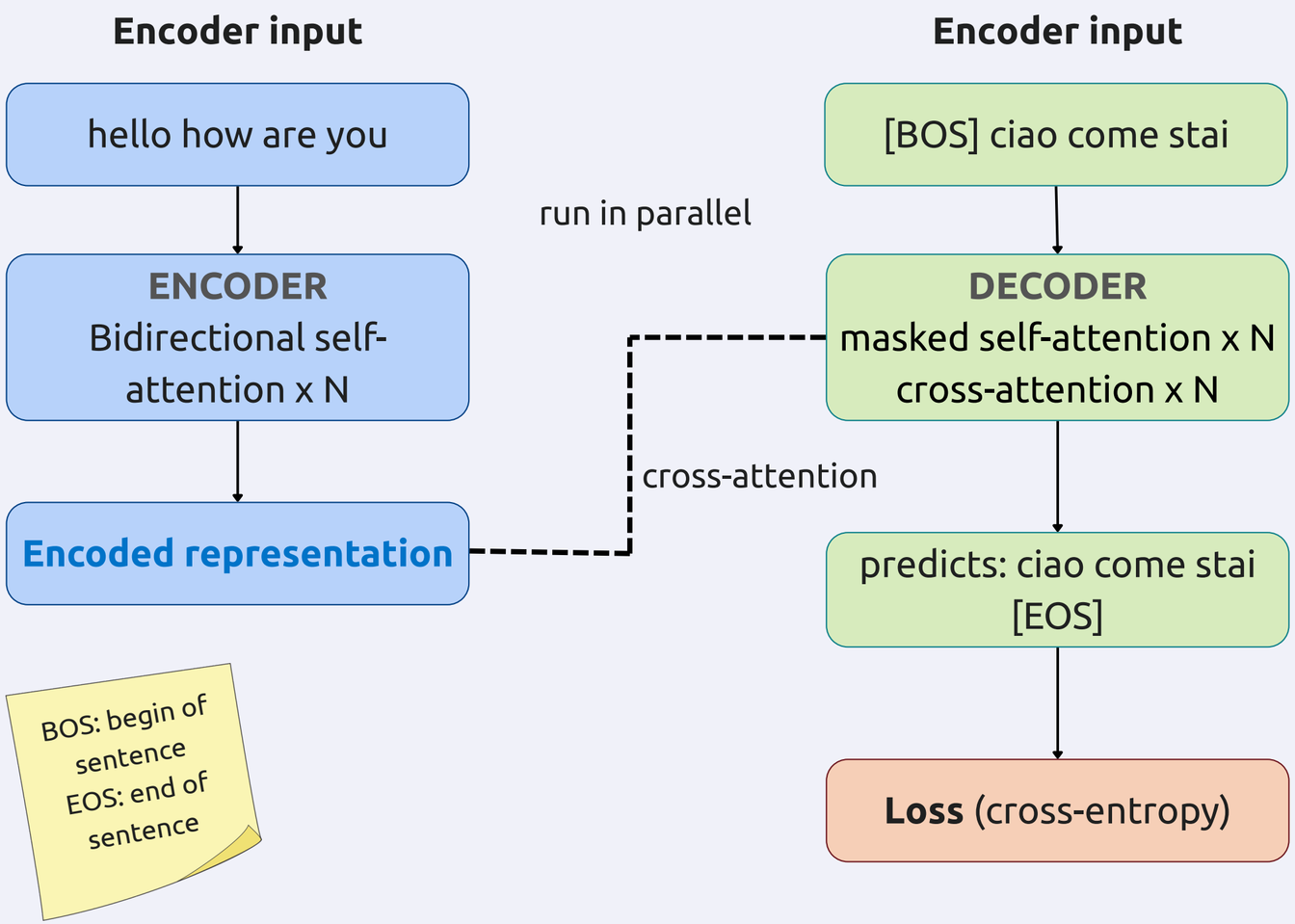
Why Cross-attention?

Input: "summarize: Scientists at the University of Tokyo have announced the discovery of a new antibiotic compound that appears effective against drug-resistant bacteria in laboratory tests."

Step 1 → cross-attention on "Scientists", "Tokyo" → generates **"Researchers"**
Step 2 → cross-attention on "antibiotic", "compound" → generates **"discover"**
Step 3 → cross-attention on "drug-resistant", "bacteria" → generates **"antibiotic"**
Step 4 → cross-attention on "laboratory", "tests" → generates **"compound"**
Step 5 → [EOS]

Output: "Researchers discover antibiotic compound"

ENCODER-DECODER: training



Masked self-attention:
the decoder can only look at tokens it has already generated, not future ones. A triangular mask blocks access to the right.

Cross-attention:
the only point where encoder and decoder meet. The decoder asks: "which part of the input is most relevant for what I'm generating now?"

Shifted right (teacher forcing):
During training, the decoder receives the correct output shifted one position to the right, not its own previous predictions.

Encoder-only vs encoder-decoder

	Encoder-only	Encoder-decoder
Goal	understand text	transform text
Output	fixed size	variable length sequence
Training target	a single label	an entire sequence
Inference	one forward pass	one pass per token

Data:
Encoder-only: ("the film is great", 1)
Encoder-decoder: ("hello how are you", "ciao come stai")

The model stops generating when it produces [EOS]:
output length is not known in advance.

ENCODER-DECODER: implementation (T5)

What changes from encoder-only?

1. Two separate inputs:
encoder_input = tokenizer(source_text)
decoder_input = tokenizer(target_text, shifted right)

2. Two vocab sizes
src_vocab_size → source language
tgt_vocab_size → target language

3. Three sub-layers in the decoder:
masked self-attention → **causal mask**
cross-attention → **Q from decoder, K/V from encoder**
feedforward

4. Two sequence lengths:
MAX_INPUT_LENGTH = 512 → **encoder**
MAX_TARGET_LENGTH = 128 → **decoder (outputs are shorter)**

Key differences in data preparation:

Encoder-only (data.py):
input_ids → [CLS] tokens [PAD]
attention_mask → [1, 1, 1, 0, 0]
label → 0 or 1

Encoder-decoder (data.py):
input_ids → source tokens
attention_mask → source mask
labels → target tokens (shifted right handled by model)
-100 on padding positions
(CrossEntropyLoss ignores -100)

T5 unifies every task with a prefix:

"summarize: " → summarization
"translate English to Italian: " → translation
"simplify: " → simplification